

```
In [2]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import LabelEncoder
```

```
In [3]: df = pd.read_csv("car_acceptability.csv")
df
```

```
Out[3]:
```

	buying price	maintenance cost	number of doors	number of persons	lug_boot	safety	evaluation
0	vhigh	vhigh	2	2	small	med	unacc
1	vhigh	vhigh	2	2	small	high	unacc
2	vhigh	vhigh	2	2	med	low	unacc
3	vhigh	vhigh	2	2	med	med	unacc
4	vhigh	vhigh	2	2	med	high	unacc
...	...	...	...	...	...	...	...
1722	low	low	5	5	med	med	good
1723	low	low	5	5	med	high	vgood
1724	low	low	5	5	big	low	unacc
1725	low	low	5	5	big	med	good
1726	low	low	5	5	big	high	vgood

1727 rows × 7 columns

```
In [4]: df.head()
```

```
Out[4]:
```

	buying price	maintenance cost	number of doors	number of persons	lug_boot	safety	evaluation
0	vhigh	vhigh	2	2	small	med	unacc
1	vhigh	vhigh	2	2	small	high	unacc
2	vhigh	vhigh	2	2	med	low	unacc
3	vhigh	vhigh	2	2	med	med	unacc
4	vhigh	vhigh	2	2	med	high	unacc

```
In [5]: df.tail()
```

```
Out[5]:
```

	buying price	maintenance cost	number of doors	number of persons	lug_boot	safety	evaluation
1722	low	low	5	5	med	med	good
1723	low	low	5	5	med	high	vgood
1724	low	low	5	5	big	low	unacc
1725	low	low	5	5	big	med	good
1726	low	low	5	5	big	high	vgood

```
In [6]: df.shape
```

```
Out[6]: (1727, 7)
```

```
In [7]: df.isnull()
```

```
Out[7]:
```

	buying price	maintenance cost	number of doors	number of persons	lug_boot	safety	evaluation
0	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...
1722	False	False	False	False	False	False	False
1723	False	False	False	False	False	False	False
1724	False	False	False	False	False	False	False
1725	False	False	False	False	False	False	False
1726	False	False	False	False	False	False	False

1727 rows × 7 columns

```
In [8]: df.columns
```

```
Out[8]: Index(['buying price', 'maintenance cost', 'number of doors',
              'number of persons', 'lug_boot', 'safety', 'evaluation'],
              dtype='object')
```

```
In [9]: df.describe()
```

```
Out[9]:
```

	number of doors	number of persons
count	1727.000000	1727.000000
mean	3.500869	3.667632
std	1.118098	1.247296
min	2.000000	2.000000
25%	3.000000	2.000000
50%	4.000000	4.000000
75%	4.500000	5.000000
max	5.000000	5.000000

```
In [10]: df.dtypes
```

```
Out[10]: buying price      object
maintenance cost    object
number of doors      int64
number of persons    int64
lug_boot             object
safety               object
evaluation            object
dtype: object
```

```
In [11]: df.nunique()
```

```
Out[11]: buying price      4
maintenance cost      4
number of doors       4
number of persons     3
lug_boot              3
safety                3
evaluation             4
dtype: int64
```

```
In [12]: df['evaluation'].unique()
```

```
Out[12]: array(['unacc', 'acc', 'vgood', 'good'], dtype=object)
```

```
In [13]: df.duplicated()
```

```
Out[13]: 0      False
1      False
2      False
3      False
4      False
...
1722   False
1723   False
1724   False
1725   False
1726   False
Length: 1727, dtype: bool
```

```
In [14]: df.drop_duplicates()
```

Out[14]:

	buying price	maintenance cost	number of doors	number of persons	lug_boot	safety	evaluation
0	vhigh	vhigh	2	2	small	med	unacc
1	vhigh	vhigh	2	2	small	high	unacc
2	vhigh	vhigh	2	2	med	low	unacc
3	vhigh	vhigh	2	2	med	med	unacc
4	vhigh	vhigh	2	2	med	high	unacc
...	...	...	...	...	...	...	...
1722	low	low	5	5	med	med	good
1723	low	low	5	5	med	high	vgood
1724	low	low	5	5	big	low	unacc
1725	low	low	5	5	big	med	good
1726	low	low	5	5	big	high	vgood

1727 rows × 7 columns

In [15]: le = LabelEncoder()

```
df['buying price'] = le.fit_transform(df['buying price'])
df['maintenance cost'] = le.fit_transform(df['maintenance cost'])
df['lug_boot'] = le.fit_transform(df['lug_boot'])
df['safety'] = le.fit_transform(df['safety'])
df['evaluation'] = le.fit_transform(df['evaluation'])
```

In [16]: X = df.drop("evaluation", axis=1)
Y = df['evaluation']

In [17]: X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_

In [18]: rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, Y_train)

Out[18]:

▼ RandomForestClassifier ⓘ ?

- Parameters
- Fitted attributes

In [19]: Y_pred = rf.predict(X_test)

In [20]: print("Accuracy:", accuracy_score(Y_test, Y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(Y_test, Y_pred))

Accuracy: 0.9624277456647399

Confusion Matrix:

```
[[ 72   1   3   1]
 [  2  10   0   3]
 [  1   0 236   0]
 [  2   0   0  15]]
```

```
In [21]: import matplotlib.pyplot as plt
```

```
importance = rf.feature_importances_
```

```
plt.bar(X.columns, importance)
```

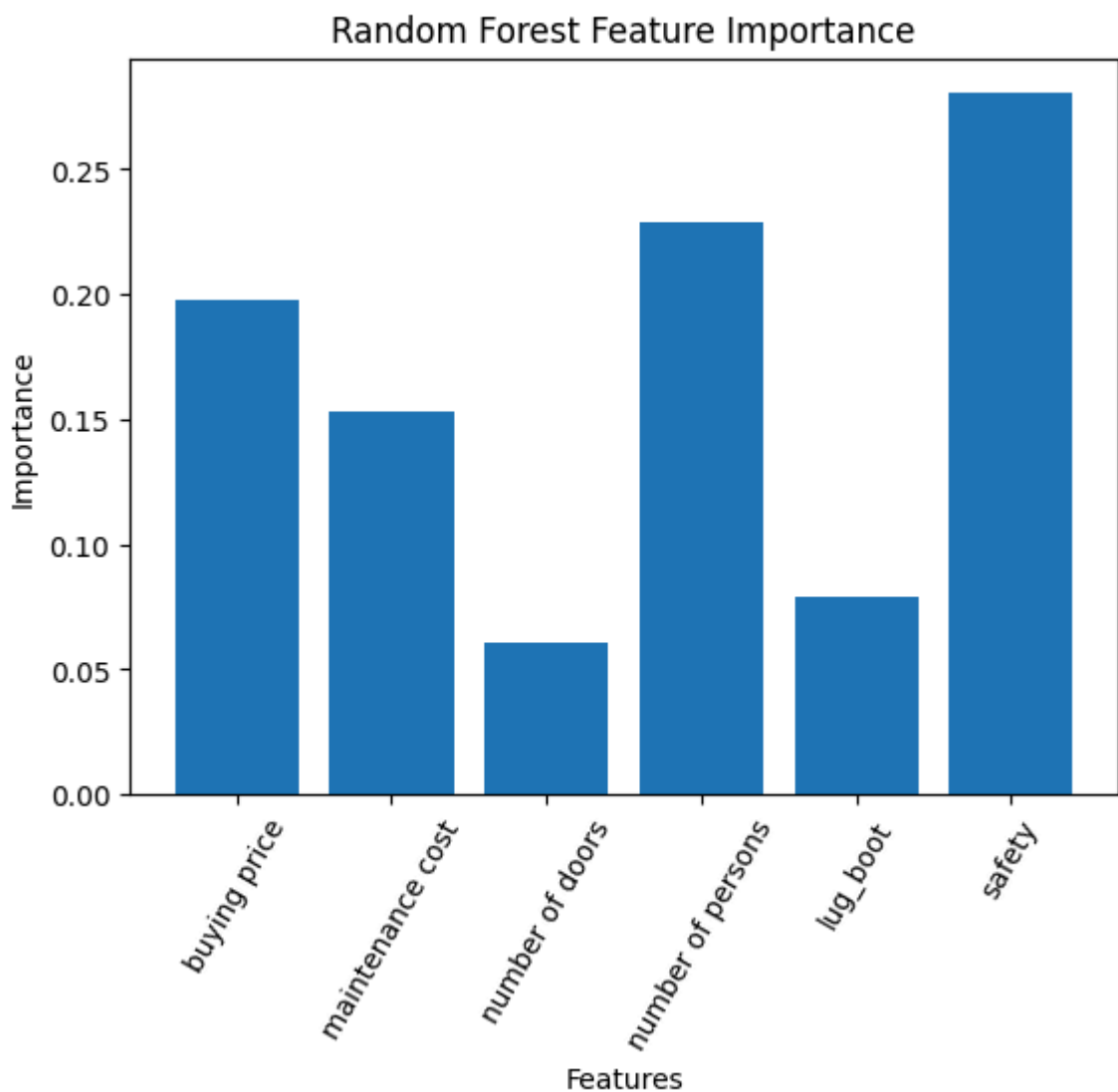
```
plt.xticks(rotation=60)
```

```
plt.xlabel("Features")
```

```
plt.ylabel("Importance")
```

```
plt.title("Random Forest Feature Importance")
```

```
plt.show()
```



```
In [ ]:
```